SECURITY ASSESSMENT

API Security Risk Analysis Report

API Under Test: JSONPlaceholder

BASE URL	https://jsonplaceholder.typicode.com
ASSESSMENT TYPE	Read-only API Security Review
TESTING APPROACH	Documentation Review, Request/Response Analysis, Header Inspection
OVERALL RISK	Medium (based on production API security expectations)

1. Executive Summary

This report presents a security risk analysis of the **JSONPlaceholder REST API**, a publicly available fake REST API service commonly used for development, testing, and prototyping.

The assessment was conducted using **non-intrusive security review techniques**, including endpoint analysis, HTTP request/response inspection, and security configuration review. No exploitation attempts, unauthorized access attempts, or destructive testing were performed.

The assessment identified that JSONPlaceholder is intentionally designed as an open testing API and therefore does not implement many security controls expected in production APIs, such as authentication, authorization, rate limiting, and user data protection mechanisms.

The primary risks identified are related to:

- Lack of authentication controls
- Absence of authorization enforcement
- Potential excessive data exposure in a production-like implementation
- Missing security headers
- Lack of visible rate limiting controls

These findings are primarily architectural observations based on the API's public design rather than confirmed vulnerabilities.

2. API Overview

API Name

JSONPlaceholder

Description

JSONPlaceholder is a free fake REST API service providing sample resources such as:

- Users
- Posts
- Comments
- Albums
- Photos
- Todos

The API is commonly used for:

- Frontend development
- API testing
- Learning REST concepts
- Prototyping applications

Example Endpoints Reviewed

Endpoint	Purpose
`/posts`	Retrieve posts
`/comments`	Retrieve comments
`/users`	Retrieve user objects
`/todos`	Retrieve task objects
`/albums`	Retrieve album data

3. Scope

In Scope

The assessment included:

- Public API endpoints
- HTTP request/response behavior
- Authentication mechanisms
- Authorization controls
- Response data exposure
- Security-related HTTP headers
- API configuration indicators

Out of Scope

The following activities were not performed:

- Exploitation attempts
- Denial-of-service testing
- Brute force testing
- Data modification
- Account compromise attempts
- Vulnerability exploitation

4. Methodology

The assessment followed a lightweight API security review methodology.

Testing Activities

1. Documentation Review

Reviewed available API behavior and endpoint structure.

2. Request/Response Analysis

Analyzed:

- HTTP methods
- Response codes
- Returned JSON objects
- Data fields

3. Header Inspection

Reviewed HTTP response headers for:

- Security headers
- Server information exposure
- Cache behavior

4. Security Control Assessment

Evaluated:

- Authentication mechanisms
- Authorization design
- Data exposure risks
- Configuration weaknesses

---

5. Tools Used

Tool	Purpose
Browser Developer Tools	Request and response inspection
Postman	API request testing
cURL	HTTP request analysis
Browser	Endpoint verification

---

6. Endpoint Analysis

6.1 Public Endpoint Review

Observation

JSONPlaceholder exposes multiple public REST endpoints without requiring authentication.

Examples:

```
GET https://jsonplaceholder.typicode.com/users

GET https://jsonplaceholder.typicode.com/posts
```

Security Assessment

For a demonstration API, public access is expected.

However, in a production API, unrestricted public access could expose sensitive resources.

Risk Rating

Informational

6.2 Authentication Requirements

Observation

The API does not require:

- API keys
- OAuth tokens
- JWT authentication
- Session cookies

Requests can be completed without identity verification.

Example:

Request:

```
GET /users
```

Response:

```
HTTP 200 OK
```

Security Assessment

Acceptable for a testing API.

For production APIs, lack of authentication could allow unauthorized users to access protected resources.

Risk Rating

Medium

---

7. Security Findings

Finding 1: Missing Authentication Controls

Risk Name:

Absence of API Authentication Mechanism

Severity:

Medium

Description:

The API allows requests without requiring authentication credentials.

No:

- API key
- Access token
- JWT token
- Session validation

was observed.

Evidence:

A request to:

```
GET /users
```

returned user data successfully without authentication headers.

Example:

```
Authorization: Not Required
Response: HTTP 200 OK
```

Business Impact:

If implemented in a real application, unauthorized users could access protected resources.

Potential impacts:

- Data exposure
- Unauthorized information retrieval
- Lack of user accountability

Recommended Fix:

For production APIs:

- Implement OAuth 2.0 or JWT authentication
- Require API keys where appropriate
- Enforce token validation
- Implement session expiration policies

Finding 2: Lack of Authorization Controls

Risk Name:

Missing Object-Level Authorization Enforcement

Severity:

Medium

Description:

The API exposes object identifiers directly:

Example:

```
/users/1
/users/2
/users/3
```

No authorization checks were observed because authentication is not implemented.

In production systems, predictable object IDs can create risks if access controls are missing.

Evidence:

Public access to:

```
GET /users/{id}
```

returns user objects without identity verification.

Business Impact:

In a real application, attackers could attempt unauthorized access to other users' resources.

Possible impacts:

- Privacy violations
- Unauthorized data access
- Regulatory compliance issues

Recommended Fix:

Implement:

- User identity verification
- Object-level authorization checks
- Role-based access control (RBAC)
- Randomized resource identifiers where appropriate

Finding 3: Potential Excessive Data Exposure

Risk Name:

API Response Contains More Data Than Required

Severity:

Low

Description:

API responses return complete objects containing multiple fields.

Example:

User endpoint response:

```
{
"id":1,
"name":"Leanne Graham",
"username":"Bret",
"email":"example@email.com",
"address":{}
}
```

For demonstration purposes this is acceptable.

However, production APIs should only return required information.

Evidence:

`/users` endpoint returns complete user objects.

Business Impact:

In real systems, unnecessary fields may expose:

- Personal information
- Internal identifiers
- Metadata

Recommended Fix:

Apply:

- Response filtering
- Data minimization
- API response schemas
- Field-level access control

Finding 4: Missing Visible Rate Limiting Controls

Risk Name:

No Observable Rate Limiting Mechanism

Severity:

Low

Description:

No rate limit headers were observed during review.

Examples:

```
X-RateLimit-Limit
X-RateLimit-Remaining
```

were not present.

Evidence:

HTTP response headers did not indicate rate limiting information.

Business Impact:

Production APIs without rate limiting may face:

- Abuse
- Automated scraping
- Resource exhaustion

Recommended Fix:

Implement:

- Request throttling
- IP-based limits
- User-based quotas
- Monitoring and alerting

Finding 5: Missing Security Header Hardening

Risk Name:

Insufficient Security Header Configuration

Severity:

Low

Description:

Security headers commonly used for API protection were not observed.

Examples include:

- Strict-Transport-Security
- Content-Security-Policy
- X-Content-Type-Options

Evidence:

Response header inspection did not show several recommended security headers.

Business Impact:

Missing headers may increase exposure to certain web-based attacks when APIs interact with browsers.

Recommended Fix:

Configure appropriate security headers:

Example:

```
Strict-Transport-Security
X-Content-Type-Options: nosniff
Content-Security-Policy
```

Finding 6: Lack of Input Validation Documentation

Risk Name:

Input Validation Controls Cannot Be Verified

Severity:

Informational

Description:

Because JSONPlaceholder is primarily a mock API, complete backend validation mechanisms cannot be confirmed.

No production database operations were tested.

Evidence:

Write operations were not assessed due to read-only testing scope.

Business Impact:

Poor validation in production systems may allow:

- Invalid data storage
- Injection attacks
- Application errors

Recommended Fix:

Production APIs should implement:

- Schema validation
- Type checking
- Input sanitization
- Backend validation rules

8. Risk Summary Table

ID	Risk	Severity
R1	Missing Authentication Controls	Medium
R2	Lack of Authorization Enforcement	Medium
R3	Potential Excessive Data Exposure	Low
R4	Missing Observable Rate Limiting	Low
R5	Missing Security Header Hardening	Low
R6	Input Validation Cannot Be Verified	Informational

9. Recommendations

Authentication

Implement:

- OAuth 2.0 authentication
- JWT-based access tokens
- API key management

Authorization

Recommended controls:

- Role-based access control
- Object-level permission checks
- Least privilege access

Data Protection

Improve security through:

- Response filtering
- Sensitive field removal
- Data classification

API Gateway Security

Consider:

- Rate limiting
- Request monitoring
- Threat detection
- Logging

Secure Configuration

Implement:

- Security headers
- HTTPS enforcement
- Error handling policies

Monitoring

Deploy:

- API access logs
- Suspicious activity detection
- Security alerts

10. Conclusion

The security review of JSONPlaceholder identified several areas where a production API implementation would require additional security controls.

Because JSONPlaceholder is intentionally designed as a public testing API, the absence of authentication and authorization mechanisms is expected behavior rather than a confirmed vulnerability.

The assessment highlights important API security practices required for real-world systems, including:

- Strong authentication
- Authorization enforcement
- Data minimization
- Secure configuration
- Rate limiting
- Proper input validation

Overall Security Assessment:

Risk Level: Medium (Based on production API security expectations)

The API is suitable for educational and testing purposes but should not be used as a model for production API security architecture without additional controls.